

Image-Processing Systolic Array Experiment Explanation

1. What This Project Is About

This project is not mainly about making an image look better. It is about modeling a hardware accelerator for a real-time image-processing service.

The core question is:

```
If an embedded camera system needs to run Gaussian blur and Sobel edge detection every frame,
what systolic array design can meet the deadline with the lowest energy?
```

The real scenario is:

```
A grayscale camera pipeline must process each frame periodically.
For real-time mode, the system must finish within 33 ms per frame.
We want the lowest-energy systolic array that still meets that deadline.
```

This matches the professor's direction: use a tangible image-processing workload, model end-to-end performance, impose time constraints, and search for an energy-efficient systolic array.

2. The Workload

The pipeline is:

```
Input grayscale image
-> Gaussian blur
-> Sobel edge detection
-> output edge image
```

Gaussian blur smooths the image. It reduces noise before edge detection.

Sobel edge detection finds edges by looking for sharp brightness changes.

This is a classic DSP and embedded-vision workload. It is simple enough to model clearly, but still meaningful because it touches every pixel in the frame.

The project tests four Gaussian kernel sizes:

```
3x3, 5x5, 7x7, 11x11
```

The bigger the kernel, the more neighboring pixels are used for each output pixel:

```
3x3    -> 9 values per output pixel
5x5    -> 25 values per output pixel
7x7    -> 49 values per output pixel
11x11  -> 121 values per output pixel
```

That is why the 11x11 case becomes much heavier than the others.

3. Why A Systolic Array

A systolic array is a grid of compute units. Each unit performs multiply-accumulate operations, often called MACs.

You can think of the array sizes like this:

```
8x8 array      -> 64 compute cells
32x32 array    -> 1024 compute cells
128x128 array  -> 16384 compute cells
```

Systolic arrays are very good at matrix multiplication. The design question is whether this image-processing workload has enough computation to keep the systolic array busy.

A larger array can be faster, but it can also waste energy if the workload is too small or poorly shaped. That is why the experiment sweeps several array sizes instead of assuming the biggest one is best.

4. Why Image Filters Are Converted To GEMM

Gaussian blur and Sobel edge detection are convolution-style operations.

For each output pixel, the filter takes a small neighborhood of input pixels, multiplies each value by a filter weight, and adds the results. That is basically a dot product.

A large collection of dot products can be represented as matrix multiplication, or GEMM.

For Gaussian blur, the project models the operation as:

```
M = number of output pixels
N = 1
K = kernel_size^2
```

For Sobel edge detection:

```
M = number of output pixels
N = 2
K = 9
```

Sobel has $N = 2$ because it computes two filters:

```
horizontal gradient
vertical gradient
```

This conversion is the bridge between image processing and systolic-array simulation.

5. Why The Project Uses Tiling

Simulating an entire frame directly would be expensive.

A 1080p frame has:

```
1920 x 1080 = 2,073,600 pixels
```

The project also sweeps many design combinations:

```
resolutions
kernel sizes
array sizes
SRAM sizes
bandwidths
dataflows
```

So instead of simulating every full frame directly, the experiment simulates representative tiles.

The main tile size is:

```
128x128
```

The image is broken into tile classes:

```
full interior tile
right-edge tile
bottom-edge tile
corner tile
```

Most tiles are full interior tiles. Edge and corner tiles handle frames whose dimensions do not divide perfectly into 128x128 chunks.

After simulating each unique tile type, the project scales the results to the full image frame. This is a practical design choice: it keeps the experiment runnable while still modeling full-frame behavior.

6. What Halo Pixels Are

Image filters need neighboring pixels.

If a tile is 128x128, computing pixels near the edge of the tile requires input pixels just outside the tile. Those extra border pixels are called the halo.

For Gaussian blur:

```
halo radius = (kernel_size - 1) / 2
```

So:

```
3x3    -> radius 1
5x5    -> radius 2
7x7    -> radius 3
11x11  -> radius 5
```

For Sobel:

```
radius = 1
```

Halo pixels increase input memory traffic, but they do not increase output pixels. This matters because memory traffic affects energy.

7. What SCALE-Sim Does

SCALE-Sim is the performance simulator.

For each tile workload and hardware configuration, SCALE-Sim reports:

```
cycle count
stall cycles
utilization
SRAM reads and writes
DRAM reads and writes
```

In this project, SCALE-Sim answers:

```
How long would this workload take on this systolic array?
```

It also tells us how much memory traffic the workload causes. This gives us the performance side of the experiment.

8. What Accelergy Does In This Project

Accelergy is used for energy accounting.

This project does not use CACTI, Aladdin, or Timeloop. Those would make the project larger and harder to finish in the available time.

Instead, the project uses table-based Accelergy.

The flow is:

```
SCALE-Sim gives action counts
-> Accelergy maps those counts to components
-> Accelergy applies an Energy Reference Table
-> total energy is computed
```

The Accelergy components are separated like this:

```
accelerator.mac_array      -> MAC operations
accelerator.ifmap_sram     -> input SRAM reads
accelerator.filter_sram    -> filter SRAM reads
accelerator.ofmap_sram     -> output SRAM writes
accelerator.ifmap_dram     -> input DRAM reads
accelerator.filter_dram    -> filter DRAM reads
accelerator.ofmap_dram     -> output DRAM writes
```

This is better than one generic SRAM number and one generic DRAM number because the energy accounting stays tied to the actual SCALE-Sim access categories.

Important caveat:

```
Because the Accelergy Energy Reference Table uses fixed per-action values,
the numerical energy totals match the original analytical model.
The improvement is methodological clarity and component-level action accounting,
not silicon-calibrated energy accuracy.
```

9. Hardware Settings Swept

The project tests:

```
Resolutions:
720p, 1080p, 2048x2048
```

```
Deadlines:
33 ms, 100 ms, 200 ms
```

```
Gaussian kernels:
3x3, 5x5, 7x7, 11x11
```

```
Array sizes:
8x8, 16x16, 32x32, 64x64, 128x128
```

```
SRAM budgets:
256 KB, 1024 KB, 4096 KB
```

```
Bandwidths:
50 GB/s, 200 GB/s, 800 GB/s
```

```
Dataflows:
weight-stationary, output-stationary
```

The most important deadline is 33 ms because it corresponds to about 30 frames per second.

The 100 ms and 200 ms deadlines represent relaxed modes.

10. What Dataflow Means

The project tests two dataflows.

Weight-stationary means filter weights stay in place inside the array as much as possible. This can help when weights are reused heavily.

Output-stationary means partial output sums stay in place while the computation accumulates. This can help reduce output movement.

Different workloads favor different dataflows.

In the results:

```
smaller kernels -> output-stationary often wins
11x11 kernel    -> weight-stationary becomes better
```

This makes sense because the large Gaussian filter has more weight reuse.

11. Main Scripts

``scripts/run_sweep.py``

Runs the SCALE-Sim experiment sweep. It creates workloads, generates SCALE-Sim inputs, runs simulations, and caches results.

``scripts/summarize_results.py``

Reads SCALE-Sim outputs, scales tile results to full frames, applies the energy model, creates summary CSVs, and generates plots.

With Accelergy:

```
.venv/bin/python scripts/summarize_results.py \
  --tile-width 128 \
  --tile-height 128 \
  --energy-backend accelergy \
  --out outputs/summary_accelergy \
  --figures figures_accelergy
```

``scripts/smoke_test_scalesim.py``

Checks that SCALE-Sim works and that reports can be parsed.

``scripts/smoke_test_accelergy.py``

Checks that Accelergy can compute energy from action counts and the Energy Reference Table.

12. Main Source Modules

``src/hpc_final/workloads.py``

Creates the GEMM shapes for Gaussian and Sobel.

``src/hpc_final/tiling.py``

Breaks each frame into representative tile classes and handles halo accounting.

``src/hpc_final/scalesim_io.py``

Writes SCALE-Sim config and topology files.

``src/hpc_final/runner.py``

Runs SCALE-Sim, handles caching, and avoids repeating completed simulations.

``src/hpc_final/parser.py``

Reads SCALE-Sim report files.

``src/hpc_final/energy.py``

Computes energy using either analytical constants or Accelergy-backed component action counts.

``src/hpc_final/accelergy_backend.py``

Builds the table-based Accelergy ERT/action-count energy path.

``src/hpc_final/analysis.py``

Builds final result tables: pipeline summaries, feasible designs, Pareto frontiers, minimum-energy designs, and bottleneck summaries.

``src/hpc_final/plots.py``

Generates the final figures.

13. Output Tables

``outputs/summary_accelergy/all_runs.csv``

Stage-level results. Each row is one Gaussian or Sobel stage result.

``outputs/summary_accelergy/pipeline_runs.csv``

End-to-end frame results. Gaussian and Sobel are combined into total pipeline latency and energy.

``outputs/summary_accelergy/feasible_designs.csv``

Says whether each design meets each deadline.

``outputs/summary_accelergy/minimum_energy_designs.csv``

The most important conclusion table. It answers:

Among the designs that meet the deadline, which one uses the least energy?

``outputs/summary_accelergy/pareto_frontier.csv``

Shows designs that are not clearly dominated. A design is dominated if another design is both faster and lower-energy.

``outputs/summary_accelergy/bottleneck_summary.csv``

Shows whether Gaussian or Sobel dominates energy and latency.

``outputs/summary_accelergy/accelergy_action_counts.csv``

Shows the component action counts passed into Accelergy.

``outputs/summary_accelergy/accelergy_ERT.yaml``

Shows the table-based Accelergy energy values per action.

14. Main Results

For 1080p at a 33 ms deadline, the minimum-energy feasible designs are:

```
3x3 Gaussian:    32x32 output-stationary, 10.07 ms, 4.64 mJ
5x5 Gaussian:    32x32 output-stationary, 11.77 ms, 8.14 mJ
7x7 Gaussian:    32x32 output-stationary, 14.32 ms, 13.40 mJ
11x11 Gaussian:  128x128 weight-stationary, 7.62 ms, 29.91 mJ
```

The interpretation is:

```
For small and medium filters:
32x32 is enough and energy-efficient.
```

```
For 11x11:
the workload becomes much heavier, so 128x128 becomes worthwhile.
```

This is the central story of the project.

15. Why Gaussian Becomes The Bottleneck

Sobel stays constant. It always uses 3x3 filters.

Gaussian grows:

```
3x3   -> 9 operations per output pixel
5x5   -> 25 operations per output pixel
7x7   -> 49 operations per output pixel
11x11 -> 121 operations per output pixel
```

So as the Gaussian kernel grows, Gaussian dominates the total workload.

The results show approximately:

```
3x3 Gaussian:    about 48% of energy
5x5 Gaussian:    about 70% of energy
7x7 Gaussian:    about 81% of energy
11x11 Gaussian:  about 91% of energy
```

That means for the large-kernel case, optimizing Sobel would not change much. Gaussian is where the time and energy go.

16. Biggest Design Lesson

The main lesson is:

```
Bigger hardware is not always better.
```

A 128x128 array has much more compute capacity than a 32x32 array.

But if the workload is small, the large array may not be fully used.

For smaller filters, 32x32 often wins because it is large enough to meet the deadline but not unnecessarily large.

For 11x11, the workload is big enough that the 128x128 array becomes useful.

That is the design argument.

17. Why This Matches The Professor's Feedback

The professor wanted:

- tangible scenario
- known DSP / embedded workload
- end-to-end performance modeling
- time constraints
- energy-efficient systolic array search

This project provides that.

The tangible scenario is:

- real-time grayscale image pipeline

The end-to-end part is:

- Gaussian latency + Sobel latency = full frame pipeline latency

The time constraint is:

- 33 ms per frame

The design objective is:

- minimum energy among configurations that meet the frame deadline

18. Honest Limitations

The project should not be oversold.

Limitations:

- No real image quality evaluation
- Only grayscale 8-bit images
- Tile-based scaling instead of full-frame direct simulation
- Table-based Accelergy, not CACTI/Timeloop technology modeling
- Energy values are relative comparisons, not silicon-accurate measurements
- Some pathological SCALE-Sim cases were skipped and recorded separately

These limitations define the scope. They do not invalidate the project.

19. One-Minute Explanation

This project studies a real-time embedded image-processing pipeline using Gaussian blur followed by Sobel edge detection. We convert both filters into GEMM-style workloads so they can be simulated on systolic arrays with SCALE-Sim. To keep the experiment feasible, we simulate representative image tiles and scale the results to full frames. SCALE-Sim gives us latency, stalls, utilization, and detailed memory access counts. We then use a table-based Accelergy model to compute component-level energy from MAC, SRAM, and DRAM actions. Finally, we search for the lowest-energy hardware design that meets frame deadlines like 33 ms. The main result is that 32x32 output-stationary arrays are usually best for 3x3, 5x5, and 7x7 filters, while the heavier 11x11 Gaussian filter benefits from a 128x128 weight-stationary array. The conclusion is that accelerator size should match the workload; the biggest array is not always the most energy-efficient choice.